人工智能与深度学习

强化学习（一）

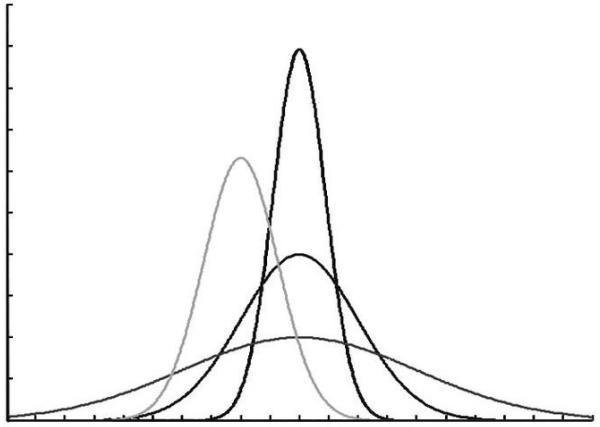
华中科技大学 机械科学与工程学院
陈伟

2026

多臂赌博机
(Multi-armed Bandits)

赌博机

- 考虑最简单的赌博机
- 按下摇臂后，产生的奖励回报服从一个固定不变、但未知的概率分布





多臂赌博机

- 多臂赌博机有 k 个摇臂，每个摇臂产生奖励的概率分布不同



- 你可以一共按下 1000 次摇臂。什么样的策略能带来最佳的预期回报?

多臂赌博机

- 动作 a: 选择哪一个摇臂并按下它

  随机变量 R：按下摇臂后得到的回报

  我们用 $q(a)$ 来表示动作 a 的实际预期回报，即

  $$q(a) = \mathbb{E}[R|a]$$

  由于每个摇臂奖励的概率分布是未知的，因此 $q(a)$ 是未知的

多臂赌博机

- 我们可以通过之前的观测来估计每个动作 a 的预期回报 $q(a)$。我们用 $Q(a)$ 来表示对预期回报的估计

  我们可以用之前动作 a 产生的回报的平均值，来估计 $Q(a)$，即

$$Q_t(a) = \frac{t \text{ 步之前采取动作 } a \text{ 的奖励总和}}{t \text{ 步之前采取动作 } a \text{ 的总次数}}$$

  每多采取一次动作 a，$Q_t(a)$ 就会更新

多臂赌博机：贪婪策略

- 我们可以基于对每个动作 a 的预期回报的估计，采用贪婪策略

$$A_t = \arg\max_a Q_t(a)$$

- **探索 vs 利用**

对于还没有探索到的动作，或者探索次数不足导致 $Q(a)$ 估计不准的动作，怎么处理？

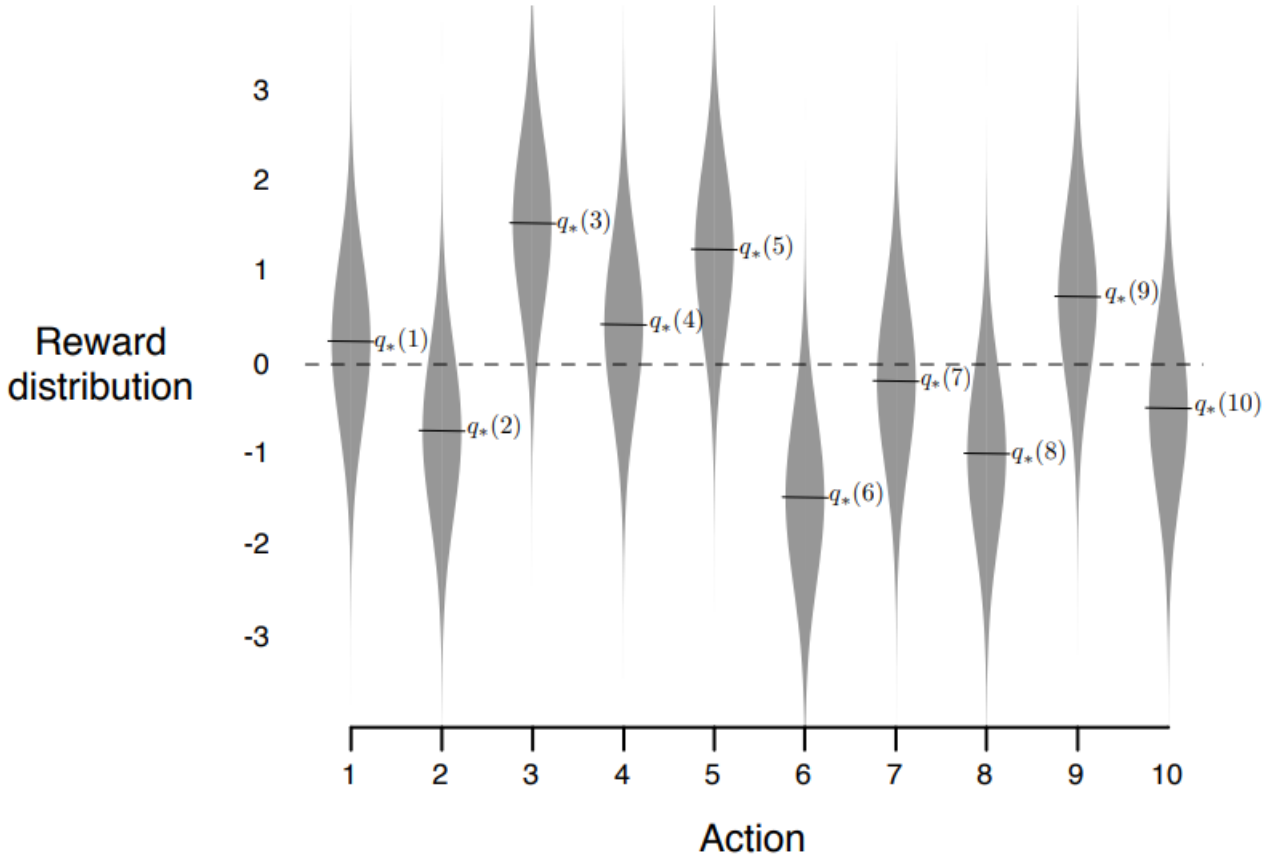
多臂赌博机： ε-贪婪策略

- **我们需要在探索和利用之间平衡**
  - 探索：对预期回报尚不明确的动作进行尝试
  - 利用：采取已知的预期回报较高的动作，以实际获得高回报

- **为了平衡探索和利用我们可以使用 ε-贪婪策略**
  - 以概率 ε 选取随机的动作
  - 以概率 1-ε 选取当前已知的最优动作，即 $\arg\max_a Q_t(a)$

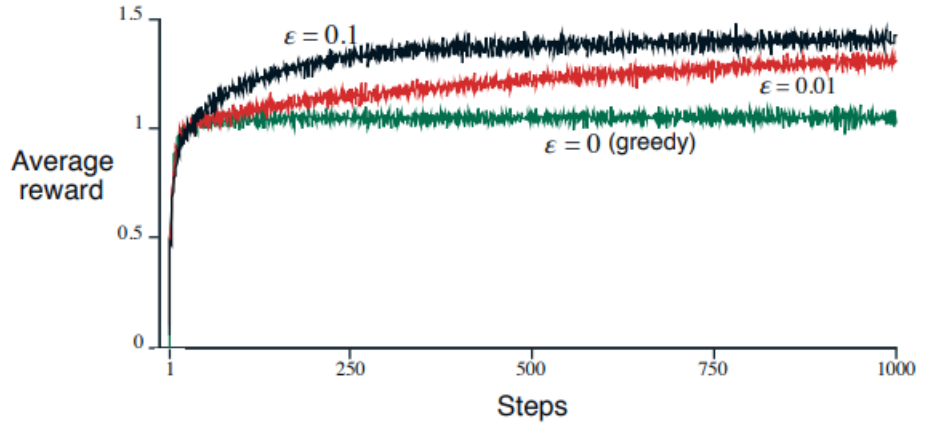
多臂赌博机： ε-贪婪策略示例

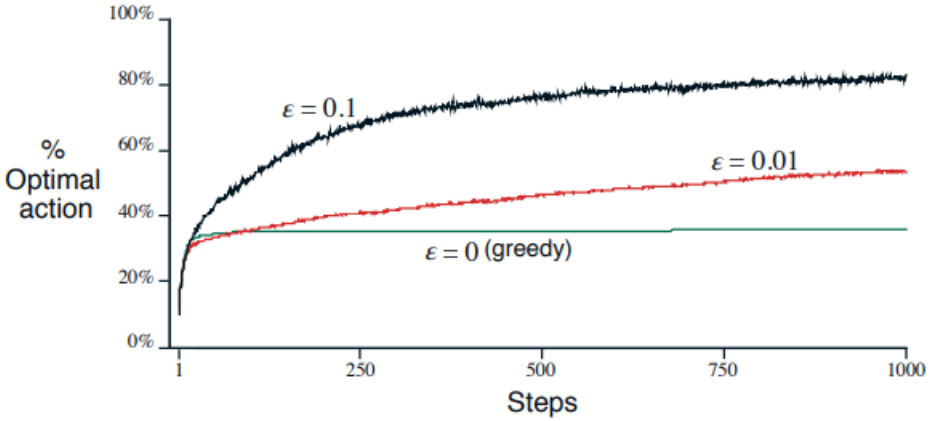
- 考虑奖励回报如下的10臂赌博机



多臂赌博机： ε-贪婪策略示例

- 采用不同 ε 值的 ε-贪婪策略





多臂赌博机： 其它策略

- 除了的 ε-贪婪策略之外，还有其它平衡探索和利用的方法吗?

多臂赌博机： 其它策略

- **上限置信区间（Upper-Confidence-Bound）策略**
  - 考虑不确定性

$$A_t = \arg\max_a \left[Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right],$$

其中，c 是常数，$N_t(a)$ 是 t 步之前采用动作 a 的次数（即取得估计值 $Q(t)$ 所有的样本数）

平方根项表示了对 $q(a)$ 估计的不确定性或方差。c 控制了探索的程度。不确定性越大，我们则认为这个动作回报的“上限”越高。

多臂赌博机： 其它策略

- **依据回报估计确定概率的策略：对每个动作都用下式确定其概率**

$$\mathbb{P}(a) = \frac{e^{-Q_t(a)/c}}{\sum_{a_i \in A} e^{-Q_t(a_i)/c}},$$

其中 c是可调整的常数

该策略使 $Q_t(a_i)$ 较大的动作有更大的概率被选中

深度 Q 学习
(Deep Q-Learning，DQN)

Q学习

- $$Q(s,a) \leftarrow Q(s,a)+\eta[R(s,a,s')+\gamma \max_{a'} Q(s',a')-Q(s,a)]$$

- 实际问题中，所有可能的 (s,a) 组合数很容易“爆炸”，远远超过计算机内存中能存放的“表格”大小

状态的数目

- **国际象棋：即使只考虑前四步，能出现的情况大约有 288,000,000,000 种**

- **围棋：3^{361}，约合 10 的 170次方**
  - 注：可观测的宇宙中，大约有10^{78}到10^{82}个原子

- **离散化后的某机器人模型： 10^{30}**

- **电子游戏图像输入：1920*1080像素，RGB 图像每个像素三个通道**

Q学习

- 使用深度神经网络函数来近似 Q 函数

$$Q(s, a; \theta) \approx Q^*(s, a),$$

其中 θ 表示深度神经网络的所有参数

- **Q学习 → 深度Q学习（DQN）**
  - Mnih, Volodymyr, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. "Playing Atari with deep reinforcement learning." arXiv preprint arXiv:1312.5602 (2013).

深度 Q 学习

- 回顾：Bellman Equation

$$Q^*(s,a) = R(s,a) + \gamma \mathbb{E}[\max_{a'} Q^*(s',a')],$$

也可写作

$$Q^*(s,a) = \mathbb{E}[r + \gamma \max_{a'} Q^*(s',a')],$$

其中 $s' \sim \mathbb{P}(\cdot|s,a)$, $r = R(s,a,s')$

深度 Q 学习

- $$Q^*(s,a) = \mathbb{E}[r + \gamma \max_{a'} Q^*(s',a')],$$

- 定义 DQN 的损失函数

$$L_i(\theta_i) = \mathbb{E}\left[(y_i - Q(s,a;\theta_i))^2\right],$$

其中 $y_i = \mathbb{E}[r + \gamma \max_{a'} Q(s',a';\theta_{i-1})]$

深度Q学习

神经网络训练需要知道损失函数 L_i 关于网络参数 θ 的梯度

$$\nabla_{\theta_i} L_i = \mathbb{E}\left[\left(r+\gamma \max_{a'} Q(s', a'; \theta_{i-1}) - Q(s, a; \theta_i)\right) \underbrace{\nabla_{\theta_i} Q(s, a; \theta_i)}\right]$$

神经网络的输出关于其参数的梯度，通过一般网络的的反向传播算法可计算

- 有了参数梯度信息后，便可用随机梯度下降的方法训练神经网络了

$$\theta_{i+1} = \theta_i - \eta \nabla_{\theta_i} L_i$$

- 每次训练，需要使用一批（s,a,r,s’）样本

深度 Q 学习示例：Atari 游戏

- 目标：在 Atari 电子游戏中取得高分

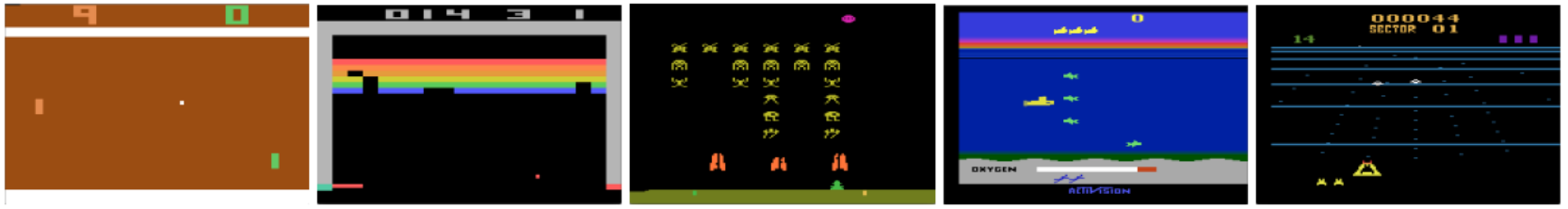


Figure 1: Screen shots from five Atari 2600 Games: (*Left-to-right*) Pong, Breakout, Space Invaders, Seaquest, Beam Rider

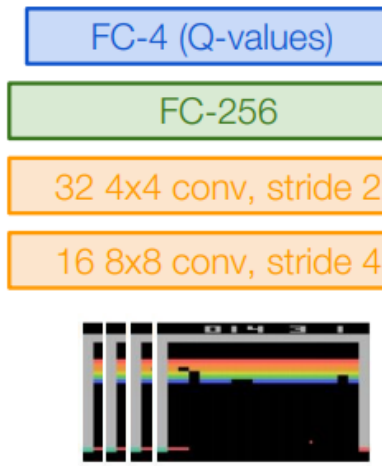
- 状态：输入的原始图像
- 动作：游戏控制，即上、下、左、右

- 奖励：游戏得分

深度 Q 学习示例：Atari 游戏

Q-network Architecture

$Q(s, a; \theta)$:
neural network
with weights θ



Last FC layer has 4-d output (if 4 actions), corresponding to $Q(s_t, a_1)$, $Q(s_t, a_2)$, $Q(s_t, a_3)$, $Q(s_t, a_4)$

Current state s_t: 84x84x4 stack of last 4 frames
(after RGB->grayscale conversion, downsampling, and cropping)

深度 Q 学习示例：Atari 游戏

灰度，下采样，裁剪

Working directly with raw Atari frames, which are 210×160 pixel images with a 128 color palette, can be computationally demanding, so we apply a basic preprocessing step aimed at reducing the input dimensionality. The raw frames are preprocessed by first converting their RGB representation to gray-scale and down-sampling it to a 110×84 image. The final input representation is obtained by cropping an 84×84 region of the image that roughly captures the playing area. The final cropping

两层隐藏 CNN，一个隐藏全连接

We now describe the exact architecture used for all seven Atari games. The input to the neural network consists is an $84 \times 84 \times 4$ image produced by ϕ. The first hidden layer convolves 16 8×8 filters with stride 4 with the input image and applies a rectifier nonlinearity [10, 18]. The second hidden layer convolves 32 4×4 filters with stride 2, again followed by a rectifier nonlinearity. The final hidden layer is fully-connected and consists of 256 rectifier units. The output layer is a fully-connected linear layer with a single output for each valid action. The number of valid actions varied between 4 and 18 on the games we considered. We refer to convolutional networks trained with our approach as Deep Q-Networks (DQN).

深度 Q 学习：经验回放

- **训练样本的选择**
  - 使用类似 mini-batch 的方法，一次使用多个（s,a,r,s’）样本来计算梯度

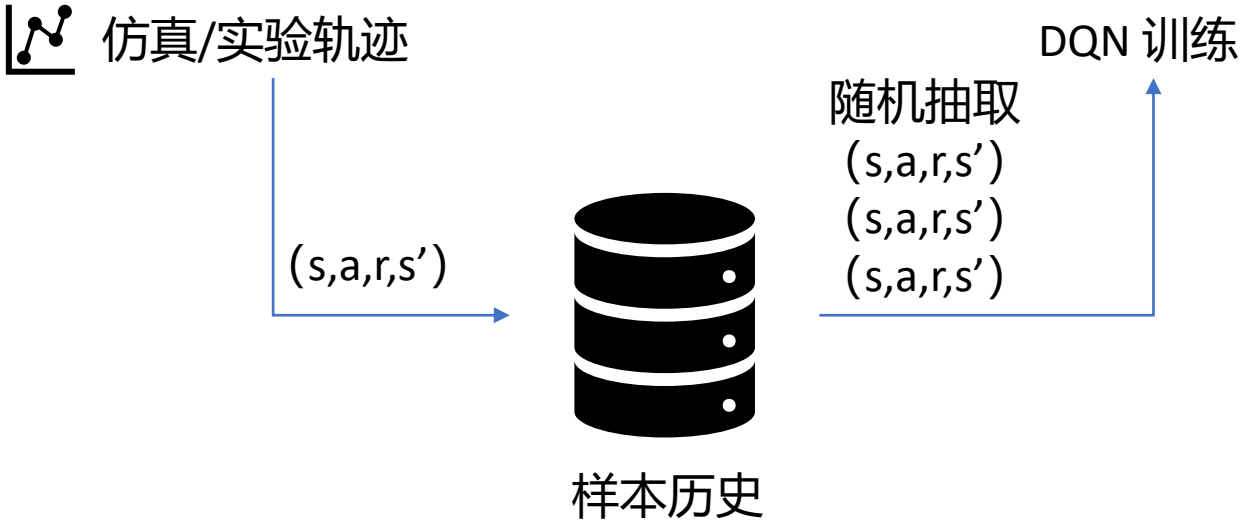
$$\nabla_{\theta_i} L_i = \mathbb{E}\left[\left(r+\gamma \max_{a'} Q(s', a'; \theta_{i-1}) - Q(s, a; \theta_i)\right)\nabla_{\theta_i} Q(s, a; \theta_i)\right]$$

  - 假如使用的是一段连续时间内的多个样本，由于这些样本之间不独立，因此训练效果不佳

  - “经验回放”（experience replay）是这个问题的解决方案

深度 Q 学习：经验回放

- **经验回放（experience replay）**
  - 将之前的（s,a,r,s’）样本历史储存下来
  - 在训练 DQN 的时候，每次随机从样本历史中选取一批样本



深度 Q 学习：策略选择

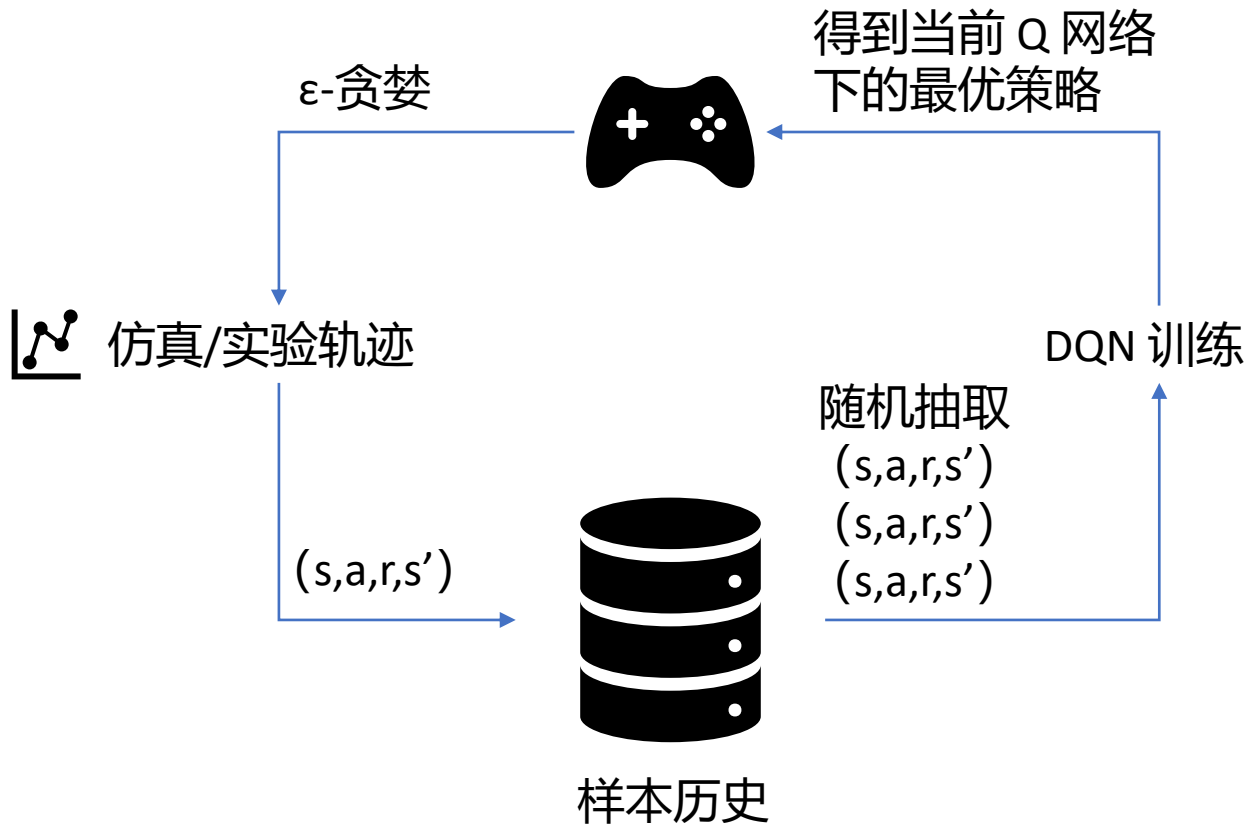
- **生成轨迹时使用的策略，需要在探索与利用之间平衡**

- **ε-贪婪策略**
  - 以概率 ε 选取随机的动作
  - 以概率 1-ε 选取当前已知的最优动作，即

$$\arg\max_{a} Q(s, a; \theta_i)$$

深度 Q 学习：策略选择

- ε-贪婪策略



深度 Q 学习：完整算法

- 初始化样本历史记忆，初始化 Q 网络

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N
Initialize action-value function Q with random weights
for episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  for $t = 1, T$ **do**
    With probability ϵ select a random action a_t
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
  end for
end for

深度 Q 学习：完整算法

- 一共进行 M 回合，即玩 M 轮游戏

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N
Initialize action-value function Q with random weights
for episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  for $t = 1, T$ **do**
    With probability ϵ select a random action a_t
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
  end for
end for

深度Q学习：完整算法

- 每回合内，初始化状态（灰度，下采样，裁剪，取4张图片历史等）

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N
Initialize action-value function Q with random weights
for episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  for $t = 1, T$ **do**
    With probability ϵ select a random action a_t
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
  end for
end for

深度Q学习：完整算法

- 使用ε-贪婪策略，观察奖励和下一步的画面

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N
Initialize action-value function Q with random weights
for episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  for $t = 1, T$ **do**
    With probability ϵ select a random action a_t
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
  end for
end for

深度Q学习：完整算法

- 根据观察，生成状态量（灰度，下采样，裁剪，取4张图片历史等）

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N
Initialize action-value function Q with random weights
for episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  for $t = 1, T$ **do**
    With probability ϵ select a random action a_t
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
  end for
end for

深度Q学习：完整算法

- 将（s,a,r,s’）存入样本历史

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N
Initialize action-value function Q with random weights
for episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  for $t = 1, T$ **do**
    With probability ϵ select a random action a_t
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
  end for
end for

深度Q学习：完整算法

- 经验回放：从样本历史中选取 mini-batch，用于本步的训练

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N
Initialize action-value function Q with random weights
for episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  for $t = 1, T$ **do**
    With probability ϵ select a random action a_t
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
  end for
end for

深度 Q 学习：完整算法

- 根据损失函数，计算梯度；使用随机梯度下降，更新 Q 网络的参数

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N
Initialize action-value function Q with random weights
for episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  for $t = 1, T$ **do**
    With probability ϵ select a random action a_t
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
  end for
end for

策略梯度（Policy Gradient），REINFORCE 算法，与 Actor-Critic 算法

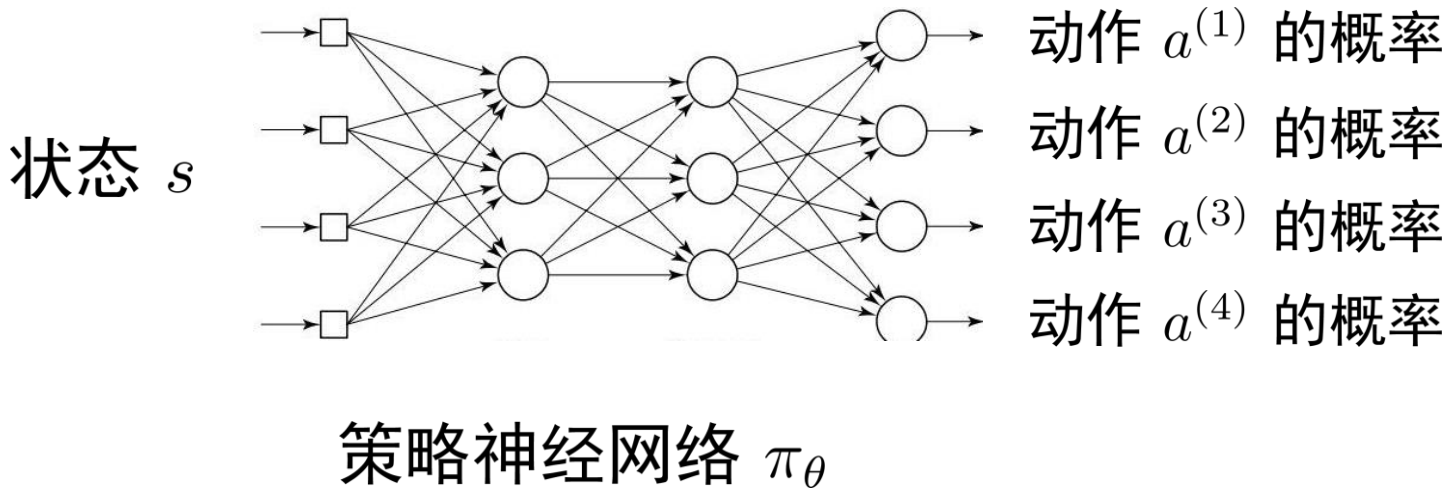
用神经网络近似 Q 函数 vs 近似最优策略

- 有些问题的 Q 函数非常复杂，而其最优策略却相对简单

- 有没有可能跳过学习 Q 函数的步骤，直接学习最优策略呢?

用神经网络近似最优策略

- 考虑基于神经网络参数化的策略 π_θ（有时也写作 $\pi_\theta(a|s)$）；该策略神经网络的输入是状态 s，输出是每个动作 a 的概率



策略神经网络

- 对于参数为 θ 的策略 π_θ，其价值函数为

$$V^{\pi_\theta}(s) = \mathbb{E}\Big[\sum_{t\geq 0}\gamma^t r_t \Big| s_0 = s, \pi_\theta(\cdot)\Big]$$

我们固定 s，那么 $V^{\pi_\theta}(s)$ 就是策略神经网络参数 θ 的函数，我们把它写作

$$J(\theta) = \mathbb{E}\Big[\sum_{t\geq 0}\gamma^t r_t \Big| \pi_\theta\Big]$$

策略神经网络

- $$J(\theta) = \mathbb{E}\Big[\sum_{t\geq 0}\gamma^t r_t \Big| \pi_\theta\Big]$$

  我们希望找到最优的策略神经网络参数 θ^*

  $$\theta^* = \arg\max_\theta J(\theta)$$

  如何找最优的 θ^* 呢？我们可以利用梯度信息，然后进行梯度下降

REINFOCE 算法

- $J(\theta) = \mathbb{E}\left[\sum_{t\geq 0}\gamma^t r_t \middle| \pi_\theta\right]$ 的梯度 $\nabla_\theta J(\theta)$ 是什么？

REINFORCE 算法告诉我们，该梯度是可以计算的，

$$\nabla_\theta J(\theta) = \mathbb{E}\left[G_t \nabla_\theta \log \pi_\theta(a_t|s_t)\right],$$

其中 $G_t = r_{t+1} + \gamma r_{t+2} + \ldots$ 是 t 步以后轨迹的奖励

REINFOCE 算法

- 因此，在取得了轨迹样本之后，这个梯度可以用下式估计

$$\nabla_\theta J(\theta) \approx G_t \nabla_\theta \log \pi_\theta(a_t|s_t),$$

估计出梯度之后，我们就可以通过梯度下降更新 θ，使之接近其最优值 θ^*

$$\theta \leftarrow \theta + \eta \nabla_\theta J(\theta)$$

REINFOCE 算法

- 上页的算法中，我们用了样本

$$G_t \nabla_\theta \log \pi_\theta(a_t|s_t),$$

来估计实际的梯度

$$\nabla_\theta J(\theta) = \mathbb{E}\left[G_t \nabla_\theta \log \pi_\theta(a_t|s_t)\right],$$

这个估计的方差很大，还有提升的空间

REINFOCE 算法

- 我们可以用一个基线（baseline） $b(s_t)$ 来降低估计的方差

$$\nabla_\theta J(\theta) \approx [G_t - b(s_t)] \nabla_\theta \log \pi_\theta(a_t|s_t),$$

简单的 $b(s_t)$ 诸如目前所有轨迹的平均奖励，就可以起到一定的效果

改进 REINFOCE 算法

- $$\nabla_\theta J(\theta) \approx [G_t - b(s_t)] \nabla_\theta \log \pi_\theta(a_t|s_t),$$

  状态 s 的价值函数 $V^{\pi_\theta}(s)$ 也可以作为 $b(s_t)$

- 事实上，我们也希望用 Q 函数来估计策略更新时所用的梯度

$$\nabla_\theta J(\theta) \approx [Q^{\pi_\theta}(s_t, a_t) - V^{\pi_\theta}(s_t)] \nabla_\theta \log \pi_\theta(a_t|s_t),$$

Actor-Critic 算法

- 而在进行策略梯度算法时，Q 和 V 都是未知的，怎么办？

  我们可以结合策略梯度方法，和 Q 学习。这样得到的算法就叫 Actor-Critic 算法

- **Actor 指的是能执行动作的策略**
- **Critic 指的是能对动作做出的评价的 Q 函数**

Actor-Critic 算法

- 在 Actor-Critic 算法中，用到了两个神经网络
  - 策略神经网络表示策略 $\pi_\theta(\cdot)$，其参数为 θ
  - 价值神经网络表示价值函数 $V_\phi(\cdot)$，其参数为 ϕ

- 我们需要估计两个梯度，用以更新 θ 和 ϕ

Actor-Critic 算法

- 对于策略神经网络，我们就使用策略梯度方法中得到的方法，其中 baseline 则使用价值神经网络估计得到的价值 $V_\phi(s_t)$

$$\nabla_\theta J \approx [G_t - V_\phi(s_t)] \nabla_\theta \log \pi_\theta(a_t|s_t),$$

- 对于价值函数神经网络，我们参考在深度 Q 学习中用到的方法

Actor-Critic 算法

回顾：深度 Q 学习中计算 Q 函数神经网络参数的公式

$$\nabla_{\theta_i} L_i = \mathbb{E}\left[\left(r+\gamma \max_{a'} Q(s', a'; \theta_{i-1}) - Q(s, a; \theta_i)\right) \nabla_{\theta_i} Q(s, a; \theta_i)\right]$$

- 对于价值函数神经网络，我们也可以采用类似上述深度 Q 学习中的方法近似梯度

$$\nabla_\phi L \approx [r + \gamma V_\phi(s') - V_\phi(s)] \nabla_\phi V_\phi(s)$$

Actor-Critic 算法

- **在 Actor-Critic 算法中，我们根据上述两个梯度估计，使用梯度下降更新策略神经网络和价值神经网络，直到最终的到一个不错的策略**

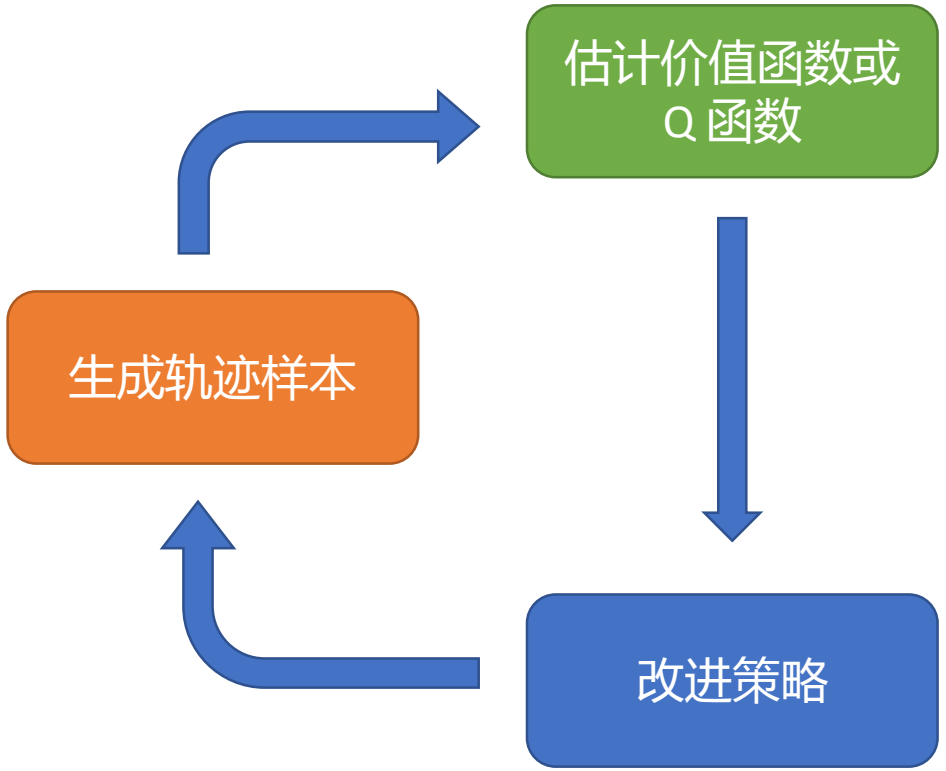
- **基于 Actor-Critic 框架，有很多变体，比如**
  - 在 Critic 中使用 Q 函数而不是价值函数
  - 两个神经网络使用不同的更新节奏
  - 使用特定方法得到的轨迹样本估计梯度
  - ……

强化学习算法的分类

分类方式一

- **是否主要基于价值函数和策略梯度**
  - 仅基于价值函数，如 Q 学习
  - 仅基于策略梯度，如 REINFORCE
  - 价值函数与策略梯度混合（Actor-Critic）

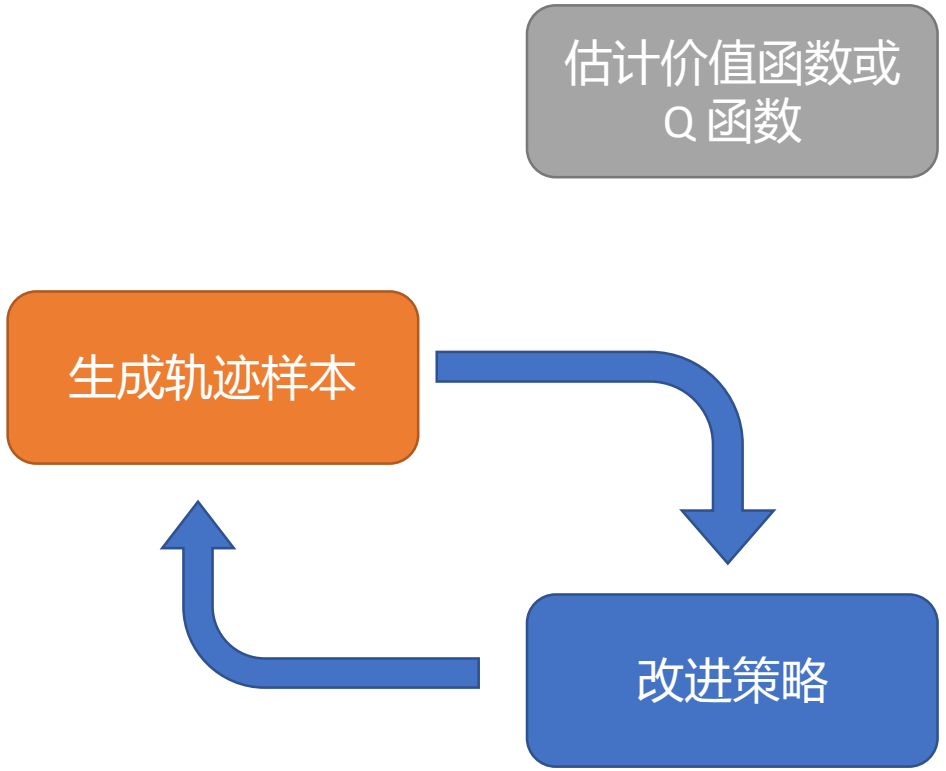
仅基于价值函数的方法



近似 $V(s)$ 或者 $Q(s,a)$

令 $\pi(s) = \arg\max_a Q(s,a)$

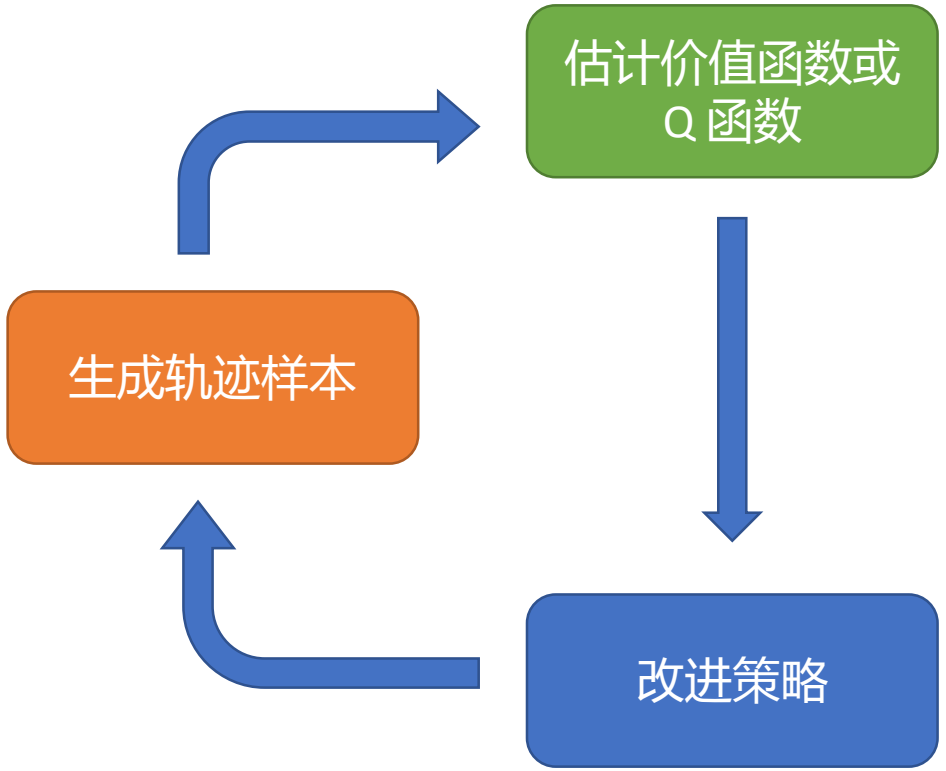
仅基于策略梯度的方法



对策略 π_θ

$$\theta \leftarrow \theta + \eta \nabla_\theta J(\theta)$$

Actor-Critic：价值函数与策略梯度混合



近似 $V(s)$ 或者 $Q(s,a)$

辅助估计

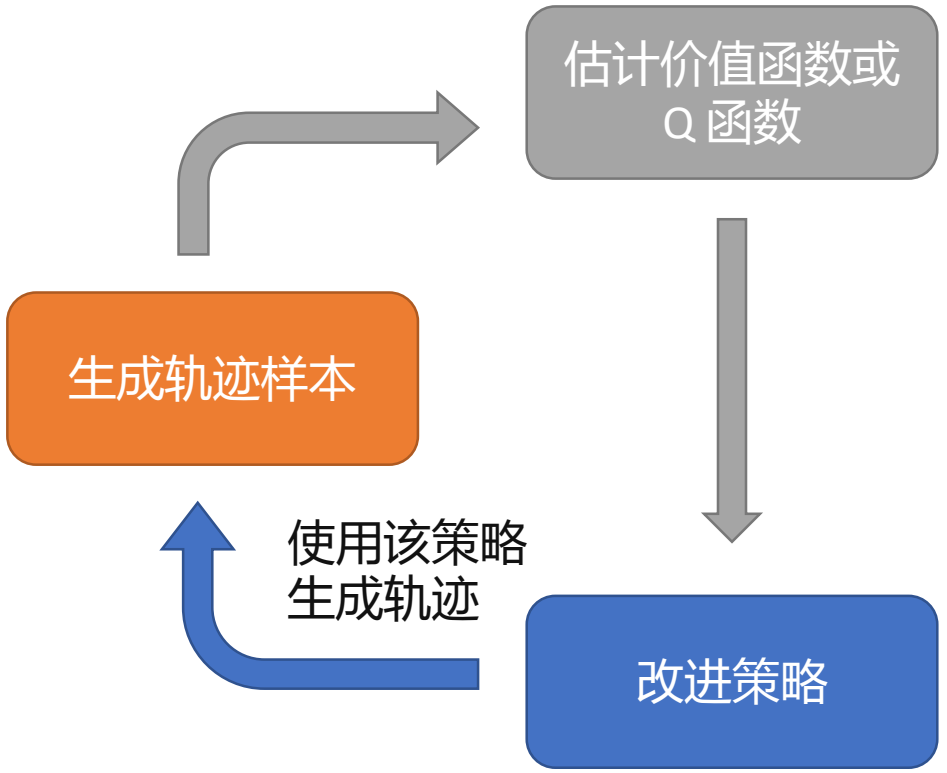
对策略 π_θ

$\theta \leftarrow \theta + \eta \nabla_\theta J(\theta)$

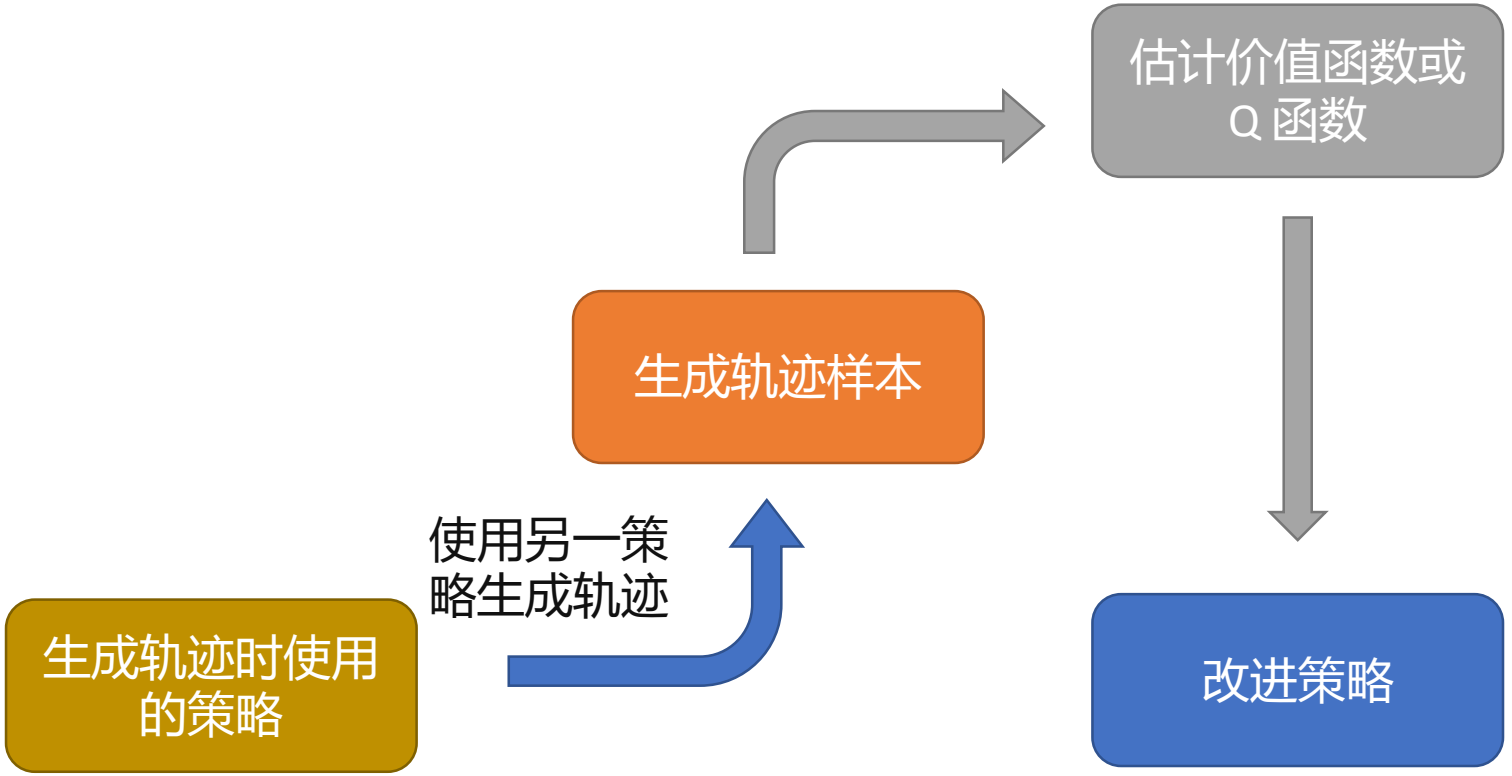
分类方式二

- **如何生成轨迹**
  - On-policy
  - Off-policy

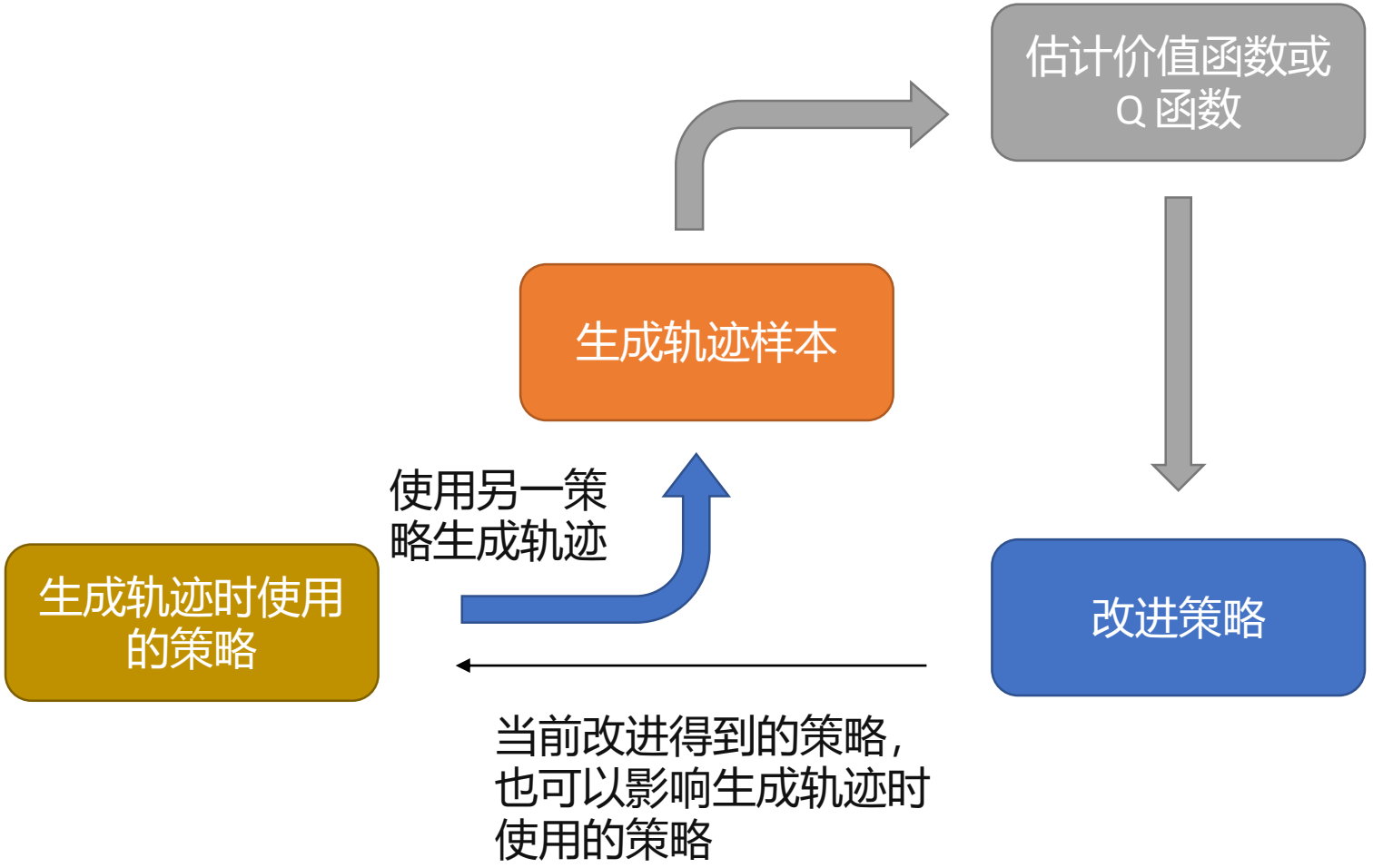
On-policy



Off-policy



Off-policy


估计价值函数或
Q 函数
生成轨迹样本
使用另一策
略生成轨迹
生成轨迹时使用
的策略
改进策略
当前改进得到的策略，
也可以影响生成轨迹时
使用的策略

AlphaGo

AlphaGo 论文

- Silver, David, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser et al. "Mastering the game of Go with deep neural networks and tree search." *nature* 529, no. 7587 (2016): 484.

Mastering the Game of Go with Deep Neural Networks and Tree Search

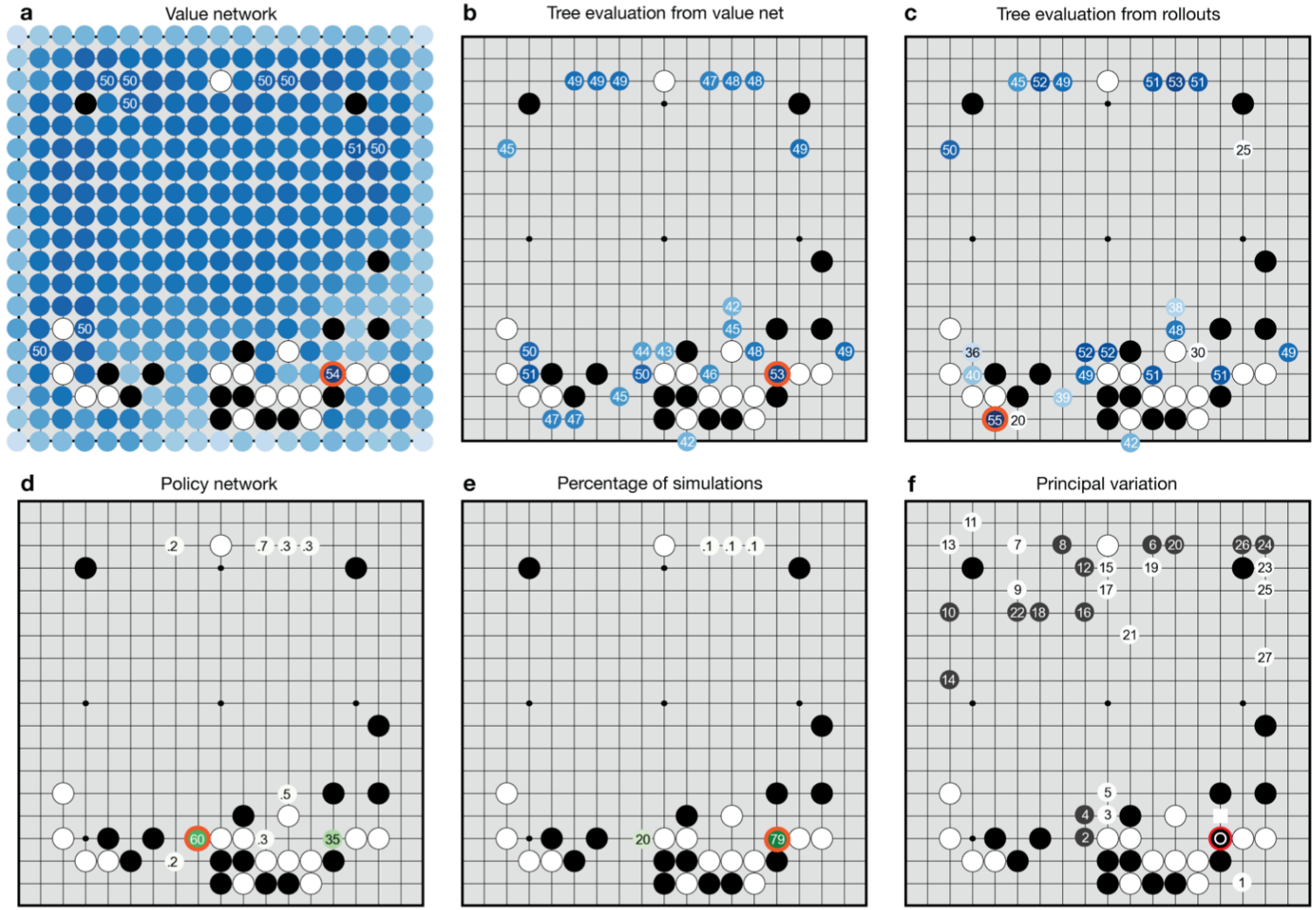
David Silver[1]*, Aja Huang[1]*, Chris J. Maddison[1], Arthur Guez[1], Laurent Sifre[1], George van den Driessche[1], Julian Schrittwieser[1], Ioannis Antonoglou[1], Veda Panneershelvam[1], Marc Lanctot[1], Sander Dieleman[1], Dominik Grewe[1], John Nham[2], Nal Kalchbrenner[1], Ilya Sutskever[2], Timothy Lillicrap[1], Madeleine Leach[1], Koray Kavukcuoglu[1], Thore Graepel[1], Demis Hassabis[1].

[1] Google DeepMind, 5 New Street Square, London EC4A 3TW.
[2] Google, 1600 Amphitheatre Parkway, Mountain View CA 94043.

*These authors contributed equally to this work.

AlphaGo 论文

- **AlphaGo使用了价值神经网络和策略神经网络**



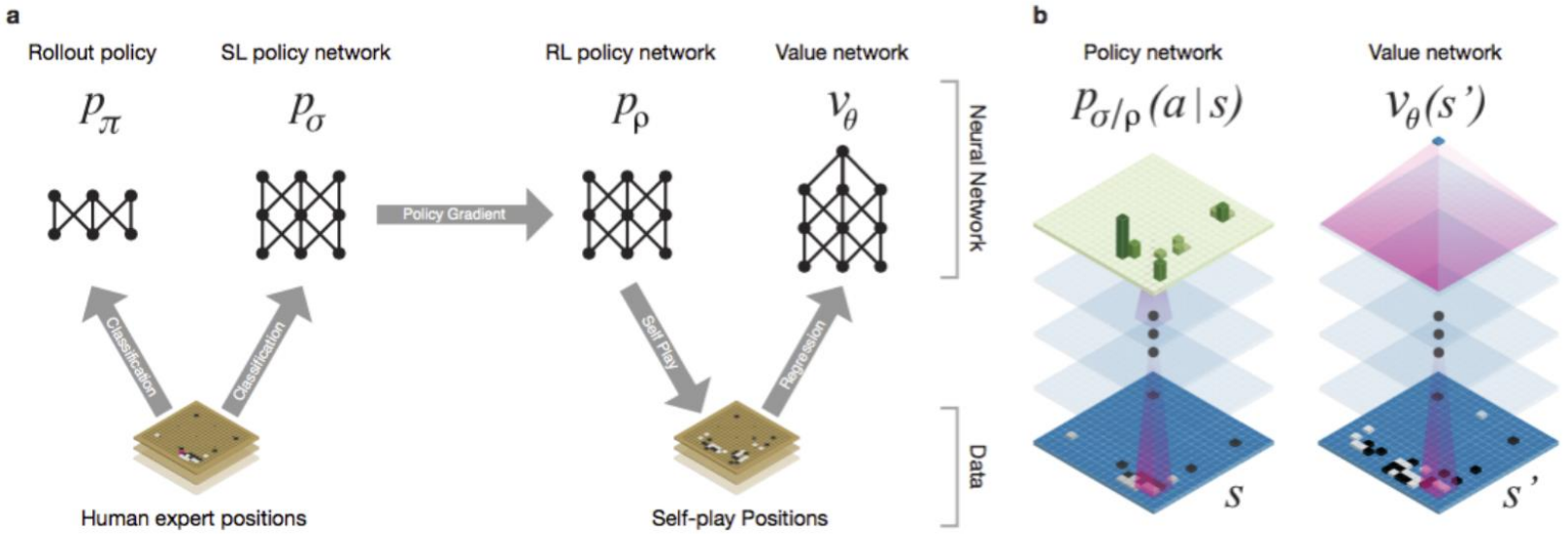
AlphaGo 论文

- **AlphaGo使用了价值神经网络和策略神经网络**

The game of Go has long been viewed as the most challenging of classic games for artificial intelligence due to its enormous search space and the difficulty of evaluating board positions and moves. We introduce a new approach to computer Go that uses *value networks* to evaluate board positions and *policy networks* to select moves. These deep neural networks are trained by a novel combination of supervised learning from human expert games, and reinforcement learning from games of self-play. Without any lookahead search, the neural networks play Go at the level of state-of-the-art Monte-Carlo tree search programs that simulate thousands of random games of self-play. We also introduce a new search algorithm that combines Monte-Carlo simulation with value and policy networks. Using this search algorithm, our program *AlphaGo* achieved a 99.8% winning rate against other Go programs, and defeated the European Go champion by 5 games to 0. This is the first time that a computer program has defeated a human professional player in the full-sized game of Go, a feat previously thought to be at least a decade away.

AlphaGo 论文

- 先使用高水平棋手对局的策略，进行有监督学习
- 再使用强化学习，让 AlphaGo 和自己下棋



AlphaGo 论文

- **策略神经网络的结构**
  - 手动提取了部分特征
  - 基于卷积神经网络

Neural Network Architecture The input to the policy network is a $19 \times 19 \times 48$ image stack consisting of 48 feature planes. The first hidden layer zero pads the input into a 23×23 image, then convolves k filters of kernel size 5×5 with stride 1 with the input image and applies a rectifier nonlinearity. Each of the subsequent hidden layers 2 to 12 zero pads the respective previous hidden layer into a 21×21 image, then convolves k filters of kernel size 3×3 with stride 1, again followed by a rectifier nonlinearity. The final layer convolves 1 filter of kernel size 1×1 with stride 1, with a different bias for each position, and applies a softmax function. The match version of *AlphaGo* used $k = 192$ filters; Figure 2,b and Extended Data Table 3 additionally show the results of training with $k = 128, 256, 384$ filters.

The input to the value network is also a $19 \times 19 \times 48$ image stack, with an additional binary feature plane describing the current colour to play. Hidden layers 2 to 11 are identical to the policy network, hidden layer 12 is an additional convolution layer, hidden layer 13 convolves 1 filter of 1×1 with stride 1, and hidden layer 14 is a fully connected linear layer with 256 rectifier units. The output layer is a fully connected linear layer with a single *tanh* unit.

AlphaGo 论文

- **策略神经网络和策略梯度**

2 Reinforcement Learning of Policy Networks

The second stage of the training pipeline aims at improving the policy network by policy gradient reinforcement learning (RL) [25,26]. The RL policy network p_ρ is identical in structure to the SL policy network, and its weights ρ are initialised to the same values, $\rho = \sigma$. We play games between the current policy network p_ρ and a randomly selected previous iteration of the policy network. Randomising from a pool of opponents stabilises training by preventing overfitting to the current policy. We use a reward function $r(s)$ that is zero for all non-terminal time-steps $t < T$. The *outcome* $z_t = \pm r(s_T)$ is the terminal reward at the end of the game from the perspective of the current player at time-step t: $+1$ for winning and -1 for losing. Weights are then updated at each time-step t by stochastic gradient ascent in the direction that maximizes expected outcome [25],

$$\Delta\rho \propto \frac{\partial \log p_\rho(a_t|s_t)}{\partial \rho} z_t \,. \tag{2}$$

AlphaGo 论文

• 价值神经网络

Ideally, we would like to know the optimal value function under perfect play $v^*(s)$; in practice, we instead estimate the value function v^{p_ρ} for our strongest policy, using the RL policy network p_ρ. We approximate the value function using a *value network* $v_\theta(s)$ with weights θ, $v_\theta(s) \approx v^{p_\rho}(s) \approx v^*(s)$. This neural network has a similar architecture to the policy network, but outputs a single prediction instead of a probability distribution. We train the weights of the value network by regression on state-outcome pairs (s, z), using stochastic gradient descent to minimize the mean squared error (MSE) between the predicted value $v_\theta(s)$, and the corresponding outcome z,

$$\Delta\theta \propto \frac{\partial v_\theta(s)}{\partial\theta}(z - v_\theta(s))\,. \tag{4}$$

AlphaGo 论文

- **使用蒙特卡洛树搜索（MCTS）**

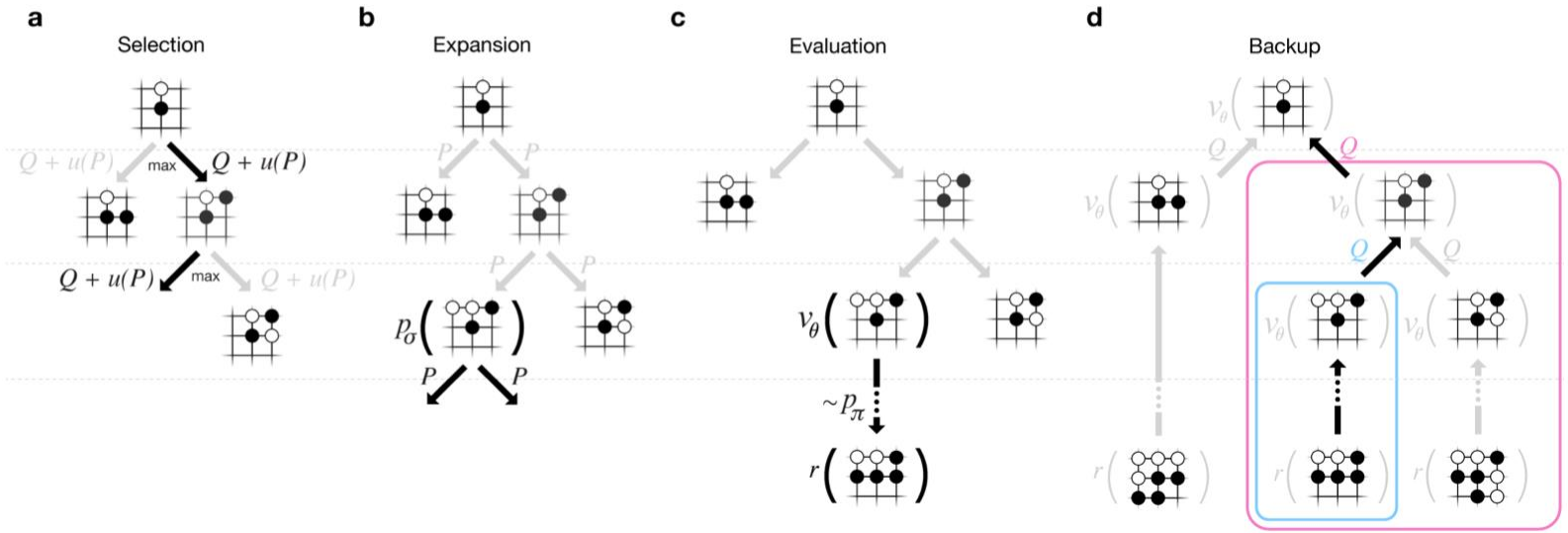
4 Searching with Policy and Value Networks

AlphaGo combines the policy and value networks in an MCTS algorithm (Figure 3) that selects actions by lookahead search. Each edge (s, a) of the search tree stores an *action value* $Q(s, a)$, *visit count* $N(s, a)$, and *prior probability* $P(s, a)$. The tree is traversed by simulation (i.e. descending the tree in complete games without backup), starting from the root state. At each time-step t of each simulation, an action a_t is selected from state s_t,

$$a_t = \underset{a}{\operatorname{argmax}} \left(Q(s_t, a) + u(s_t, a)\right), \tag{5}$$

so as to maximize action value plus a bonus $u(s, a) \propto \frac{P(s,a)}{1+N(s,a)}$ that is proportional to the prior probability but decays with repeated visits to encourage exploration. When the traversal reaches

AlphaGo 论文


a
Selection
Q + u(P)
max
Q + u(P)
Q + u(P)
max
Q + u(P)
b
Expansion
P
P
P
P
p_σ
P
P
c
Evaluation
v_θ
$\sim p_\pi$
r
d
Backup
v_θ
Q
Q
v_θ
v_θ
Q
Q
v_θ
v_θ
r
r
r

AlphaGo Zero **论文**

- Silver, David, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert et al. "Mastering the game of go without human knowledge." nature 550, no. 7676 (2017): 354-359.

- **相对 AlphaGo 有了大幅改进**
  - 不再需要使用专业棋手的棋局来做有监督学习
  - 不再使用手动提取的特征，直接使用 19*19 黑/白/空作为输入
  - 使用一套共用的神经网络前端来表示策略和价值

AlphaGo Zero 论文

The input features s_t are processed by a residual tower that consists of a single convolutional block followed by either 19 or 39 residual blocks[4].

The convolutional block applies the following modules:

(1) A convolution of 256 filters of kernel size 3×3 with stride 1

(2) Batch normalization[18]

(3) A rectifier nonlinearity

Each residual block applies the following modules sequentially to its input:

(1) A convolution of 256 filters of kernel size 3×3 with stride 1

(2) Batch normalization

(3) A rectifier nonlinearity

(4) A convolution of 256 filters of kernel size 3×3 with stride 1

(5) Batch normalization

(6) A skip connection that adds the input to the block

(7) A rectifier nonlinearity

The output of the residual tower is passed into two separate 'heads' for computing the policy and value. The policy head applies the following modules:

(1) A convolution of 2 filters of kernel size 1×1 with stride 1

(2) Batch normalization

(3) A rectifier nonlinearity

(4) A fully connected linear layer that outputs a vector of size $19^2 + 1 = 362$, corresponding to logit probabilities for all intersections and the pass move

The value head applies the following modules:

(1) A convolution of 1 filter of kernel size 1×1 with stride 1

(2) Batch normalization

(3) A rectifier nonlinearity

(4) A fully connected linear layer to a hidden layer of size 256

(5) A rectifier nonlinearity

(6) A fully connected linear layer to a scalar

(7) A tanh nonlinearity outputting a scalar in the range $[-1, 1]$

The overall network depth, in the 20- or 40-block network, is 39 or 79 parameterized layers, respectively, for the residual tower, plus an additional 2 layers for the policy head and 3 layers for the value head.

练习题

一般认为，以下强化学习算法中最适合用于连续系统控制的是_____

A. Deep Q-Learning（DQN）

B. Double Deep Q-Learning（DDQN）

C. Deep Deterministic Policy Gradient（DDPG）

D. Asynchronous Advantage Actor Critic（A3C）

练习题

一般认为，以下强化学习算法中最适合用于连续系统控制的是_____

A. Deep Q-Learning（DQN）

B. Double Deep Q-Learning（DDQN）

C. Deep Deterministic Policy Gradient（DDPG）

D. Asynchronous Advantage Actor Critic（A3C）

【答案】：C

【解析】：DDPG专为连续动作空间设计，采用Actor-Critic架构。

Actor网络直接输出连续动作值，Critic网络评估动作的Q值，

结合确定性策略梯度，是连续系统控制的经典算法，最适合连续系统控制。